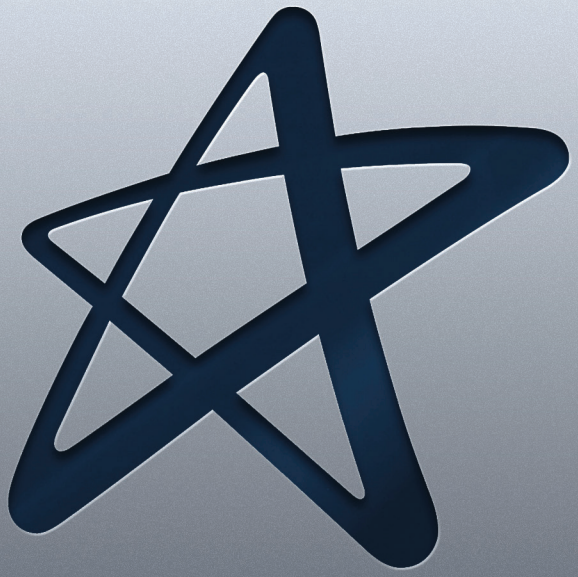
Mineração de Dados



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico

Análise de Textos e Prática de *Kmeans*

Responsável pelo Conteúdo:
Prof. Dr. Alberto Messias

Revisão Textual:
Prof.ª Dr.ª Luciene Oliveira da Costa Granadeiro

UNIDADE Análise de Textos e Prática de *Kmeans*

- Caso prático de execução do algoritmo *Kmeans*;
- Processo de Análise de Textos;
- Conversão de Documentos e Remoção de Palavras;
- Extração de Feições;
- Seleção de Feições;
- Representação de Documentos;
- Exemplo Prático de Análise de Textos com *Weka*.



OBJETIVO DE APRENDIZADO

- Introduzir um caso prático do algoritmo *kmeans*, utilizando o *software Weka*, mostrar o processo de análise de textos, bem como o método TF/IDF para seleção de feições ou palavras e, por fim, ilustrar um caso prático de análise de textos e algoritmo *Kmeans* em conjunto.

Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu "momento do estudo";
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Caso Prático de Execução do Algoritmo *Kmeans*

Vamos utilizar o *software WEKA* para fazer um experimento com a execução do algoritmo *kmeans*. O experimento será com uma base de dados *Iris Flower dataset*, comumente utilizada para ilustrar o funcionamento do algoritmo relacionado ao reconhecimento de padrões.

O *software* é um *software* livre e muito comum para processamento de bases de dados e validações experimentais, há a possibilidade de se importar arquivos, fazer conexões diretas com bancos de dados ou, ainda, gerar dados aleatórios para experimentos.

Segue a **Figura 1** com a tela do *Weka* ao se carregar o arquivo da base de dados.

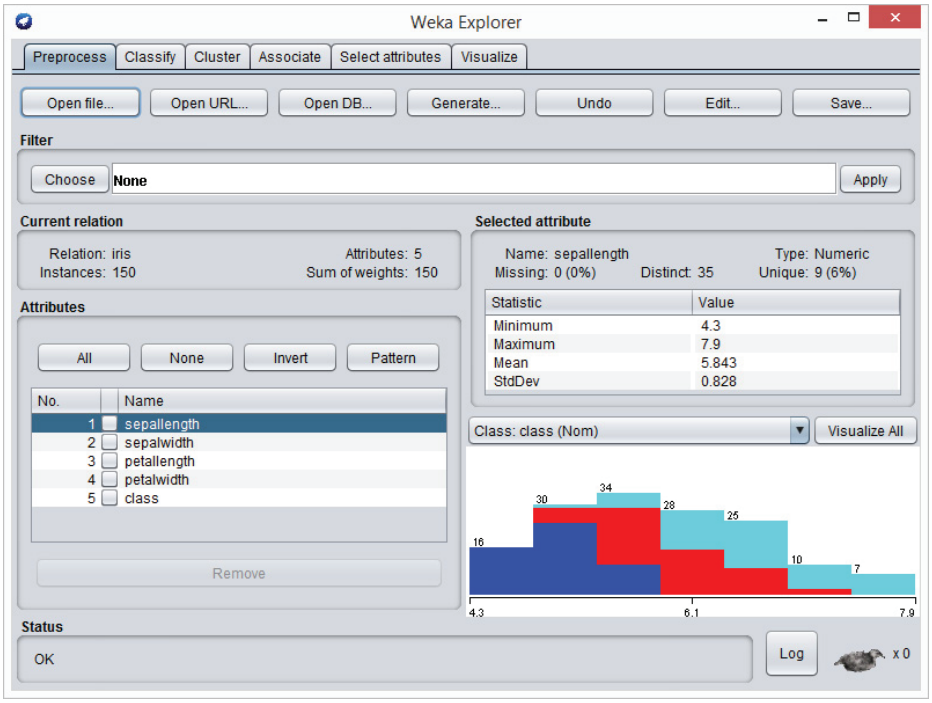


Figura 1

Na tela, é possível se observar os valores mínimos, máximos, a média e o desvio padrão, a quantidade de instâncias na base e os atributos existentes.

Cada aba do *software* possui um conjunto de algoritmos com a mesma finalidade, como algoritmos de classificação, cluster, regras de associação, seleção de atributos e visualização dos dados e resultados.

A **Figura 2** ilustra a aba de algoritmos de *clustering* – nesse caso, ao clicar no botão "*choose*", pode-se escolher o algoritmo desejado.

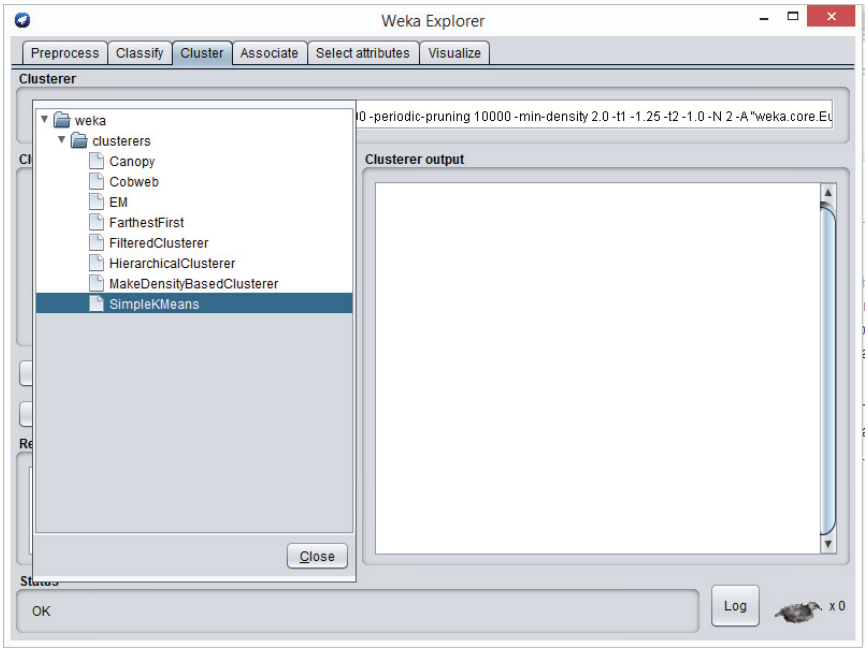


Figura 2

Nesse caso, o algoritmo selecionado é o "*SimpleKmeans*" em sua implementação principal. A **Figura 3** ilustra os parâmetros possíveis para o algoritmo *kmeans*.

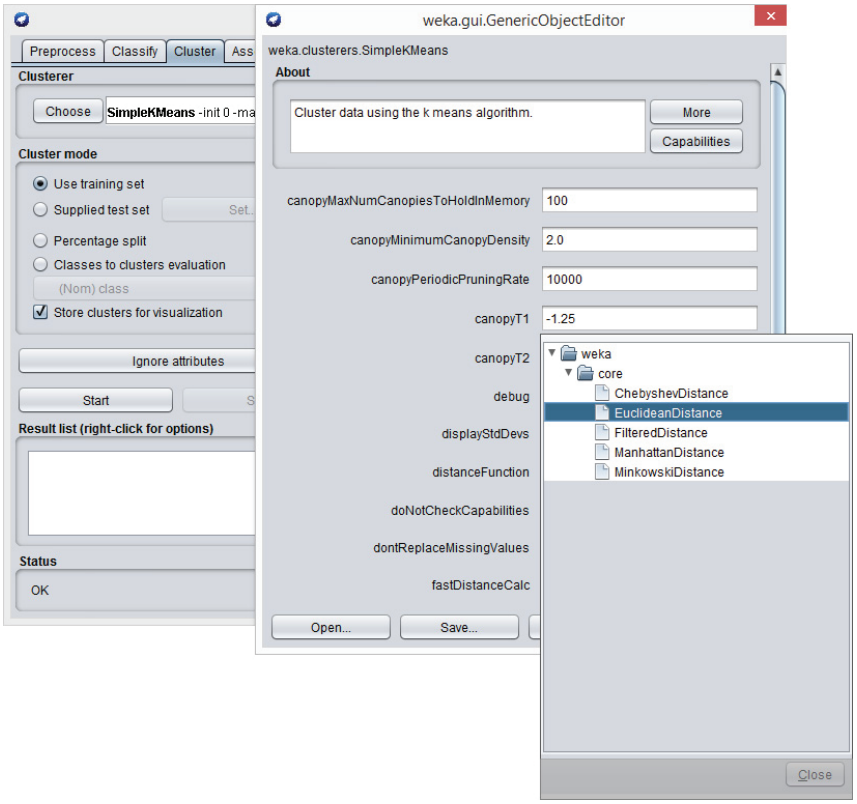


Figura 3

Na tela exibida na **Figura 3**, pode-se observar a escolha da métrica de distância, nesse caso, a distância euclidiana, dentro outros parâmetros, como, por exemplo, a quantidade de *clusters* "K" ou a quantidade máxima de iterações do algoritmo.

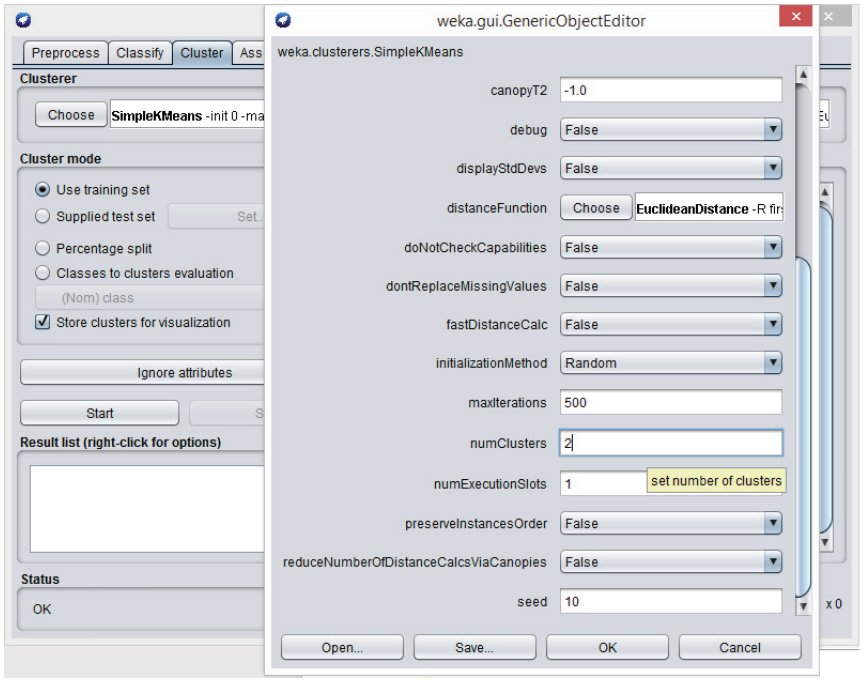


Figura 4

A listagem exibe a saída do algoritmo na tela principal de execução dos algoritmos no *Weka*: **talvez colocar essa saída em um quadro com fonte diferente.**

```
=== Run information ===

Scheme: weka.clusterers.SimpleKMeans -init 0 -max-candidates 100 -periodic-
pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 3 -A "weka.core.
EuclideanDistance -R first-last" -I 500 -num-slots 1 -S 10

Relation: iris

Instances: 150

Attributes: 5

        sepallength

        sepalwidth

        petallength

        petalwidth

        class

Test mode: evaluate on training data
```

```
=== Clustering model (full training set) ===

kMeans

======

Number of iterations: 3

Within cluster sum of squared errors: 7.817456892309574

Initial starting points (random):


Cluster 0: 6.1,2.9,4.7,1.4,Iris-versicolor

Cluster 1: 6.2,2.9,4.3,1.3,Iris-versicolor

Cluster 2: 6.9,3.1,5.1,2.3,Iris-virginica


Missing values globally replaced with mean/mode

Final cluster centroids:

                          Cluster#

Attribute           Full Data          0            1            2

                    (150.0)          (50.0)       (50.0)       (50.0)

=============================================================================

sepallength         5.8433           5.936        5.006        6.588

sepalwidth          3.054            2.77          3.418        2.974

petallength         3.7587           4.26          1.464        5.552

petalwidth          1.1987           1.326         0.244        2.026

class      Iris-setosa Iris-versicolor        Iris-setosa Iris-virginica


Time taken to build model (full training data) : 0 seconds

=== Model and evaluation on training set ===

Clustered Instances

0      50 ( 33%)

1      50 ( 33%)

2      50 ( 33%)
```

Observe, na saída do algoritmo, os pontos principais respectivamente, como:

- Os parâmetros passados na execução do algoritmo;
- A quantidade de instâncias da base de dados;
- Os atributos presentes na base;
- Número de iterações do algoritmo;
- O valor de erro quadrado do modelo gerado;
- Os centroides gerados pelo algoritmo;
- Uma tabela com as informações sobre as instâncias classificadas em cada *cluster*;
- A quantidade de instâncias em cada cluster;

A **Figura 5** ilustra o resultado das instâncias separadas em *clusters*, ao se clicar com o botão direito sobre o experimento, e ir à opção visualizar o modelo gerado, conforme a tela exibida na **Figura 4**.

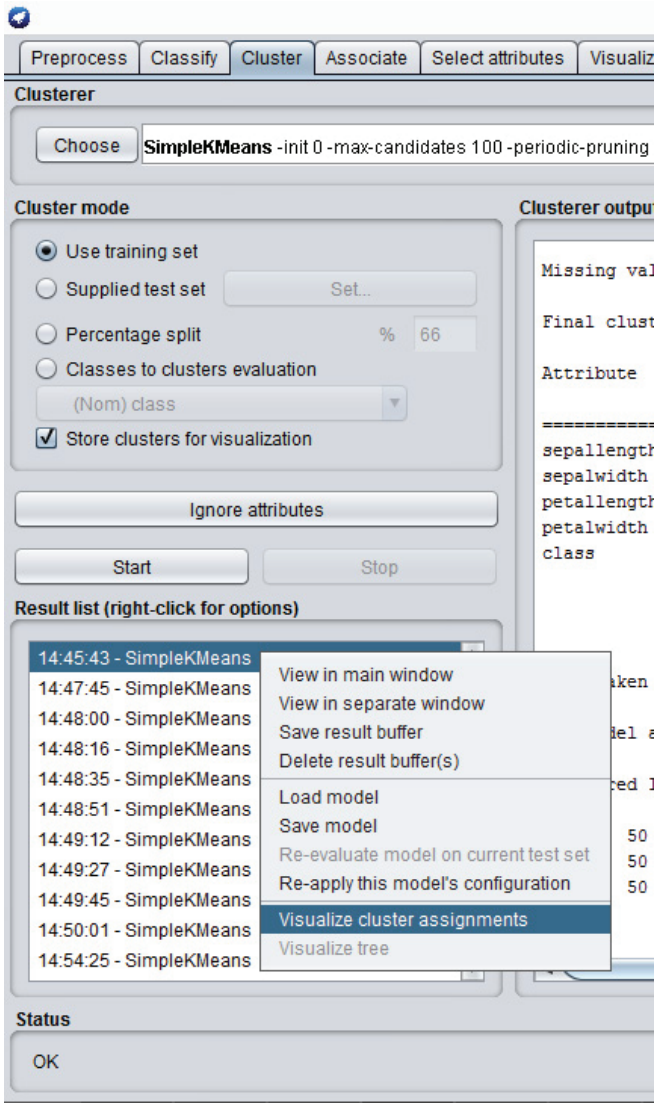


Figura 5

Clique para se exibir o modelo de *clusters* gerado.

Note que as instâncias classificadas em seus respectivos grupos estão com cores que identificam o cluster específico.

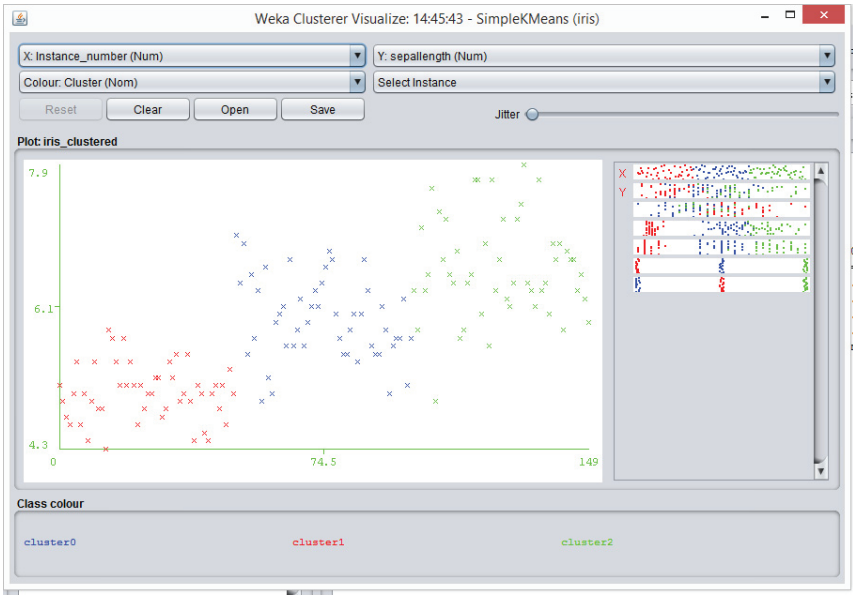


Figura 6 – Modelo de *clusters* gerado e as instâncias classificadas

Dado que na saída do algoritmo é exibido o erro quadrado do modelo, é possível então, se validar o modelo ao se executar o algoritmo com variando o número de *clusters*.

Para o experimento de verificação da minimização dos erros quadrados para a validação do modelo o algoritmo foi executado treze vezes, variando o "K" entre 1 e 10 e logo após com 50, 100 e 146 clusters, segue a tabela extraída com as somas dos erros quadrados:

Tabela 1

K – clusters	Erro Quadrado
1	141,1381720229770
2	62,1436882815797
3	7,8174568923096
4	6,6138232746904
5	6,2935568618108
6	6,1318396310806
7	5,2024143888778
8	4,8527343852958
9	4,6720734767839
10	4,6239733170433
50	0,6325772361931
100	0,1908186705859
146	1.0030885127016854E-34

Observe que, inicialmente, o valor de da soma dos erros quadrados para um único cluster traz um valor bastante alto; e, ao se aumentar o número de cluster, o modelo vai se especializando e esse erro é minimizado. Note que, ao se aproximar do número total de instância o erro tende a zero, porém, o modelo fica extremamente especializado, o que não é um bom resultado.

A **Figura 7**, ilustra graficamente o decaimento da soma dos erros quadrados.

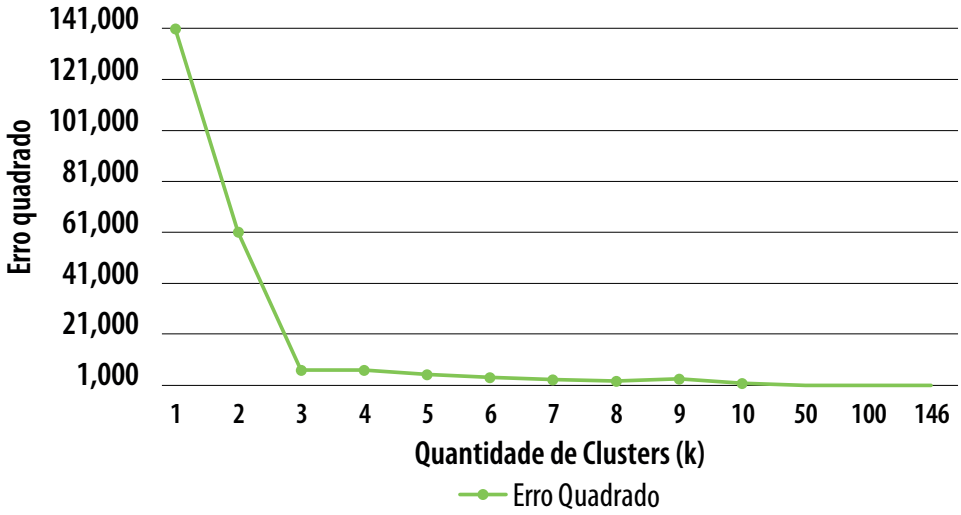


Figura 7

Conforme o informado anteriormente, a quantidade ideal de *clusters* para um dado modelo ocorre quando há um joelho no gráfico; note que o valor ideal de clusters para a base usada é de três. O erro inicialmente é bastante alto, quando se chega ao número ideal, ele cai bruscamente e, logo após a redução do erro, é pequena, e no final ele tende a zero, conforme se observa com a execução do algoritmo com 146 *clusters*, sendo que a base total possui 150 instâncias.

O *software Weka* será interessante para fazer a modelagem inicial dos conjuntos e tipos de dados a se trabalhar, antes de se implementar o modelo usando tecnologias de *Big Data*, como, por exemplo, o *Hadoop* ou *Spark*.

Vale destacar, ainda, que o *software Weka* possui uma API em linguagem de programação *Java*, que permite o uso de suas implementações algorítmicas em seus projetos *Java*, de modo a permitir uma integração mais profunda em seus projetos de BI ou mineração de dados.

> Com o modelo de clusters gerado, é possível criar ferramentas para a classificação ou agrupamento de uma nova instância ainda não agrupada. Note que, caso você tenha uma nova instância, ela poderá estar em um dado grupo, ao qual é a menor distância para o centroide do grupo, ou seja, ela estará em um determinado grupo, no qual a menor distância euclidiana dela para o centroide grupo ocorrer.

Processo de Análise de Textos

A mineração de textos é definida como um processo de extração de informações relevantes ou conhecimento a partir de textos não estruturados (HOTHO; NüRNBERGER; PAASS, 2005). O processo de categorização de documentos é uma subárea da mineração em textos, que, se definido como um processo para agrupar documentos similares, a partir da organização do conhecimento e da remoção de redundâncias e variações de palavras existentes nos documentos (BRÜCHER; KNOLMAYER; MITTERMAYER, 2002).

A **Figura 8** ilustra a arquitetura do processo de categorização de documentos.

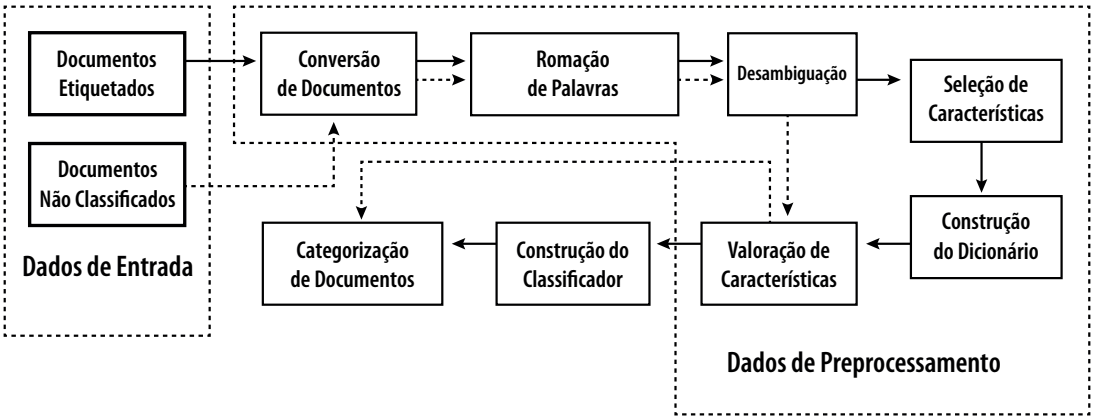


Figura 8 – Arquitetura do processo de categorização de documentos
Fonte: Adaptado de GUO *et al.*, 2003

Conversão de Documentos e Remoção de Palavras

Ao se trabalhar com diversas fontes de dados de *Big Data*, o processo inicial de requisição de dados poderá variar bastante, dependendo do tipo de fonte de dados, por exemplo, dados de redes sociais, dados da *WEB*, *Blogs*, fóruns em sistemas específicos, bases de e-mails, enfim, uma infinidade de fontes de dados que deverão ser trabalhadas em suas especificidades. Cabe ressaltar, ainda, que, dada a característica da análise, é sempre importante se conseguir etiquetar o dado com fonte de origem ou autor; o agrupamento dos documentos, seja por data, autor ou origem, poderá alterar grandemente o resultado da análise de textos e isso deve variar também de acordo com o projeto e tecnologia.

A fase inicial de dados de entrada, sejam dados etiquetados, deve ser bem definida e trabalhada em sua especificidade de projeto e finalizará com a etapa de conversão dos documentos.

Como exemplo, pode-se pensar no resgate de dados de postagens em *Blogs*, onde cada postagem deverá ser inicialmente trabalhada para se removerem as *TAGs* HTML, de modo a deixar tudo com texto plano, etiquetadas com características de autor e informações temporais. Nesse passo, hipoteticamente, ao trabalhar com a plataforma *HADOOP*, deverão ser criados diversos arquivos em texto plano e agrupados ou separados de maneira temporal ou por autor em pastas no sistema de arquivos. Note que essa separação em pastas poderá influenciar o resultado e deverá ser feito de acordo com a análise que se requer.

Algumas etapas serão agrupadas para a descrição mais adequada.

Extração de Feições

O processo de extração de feições agrupa os passos de conversão de documentos, remoção de palavras e desambiguação ilustrados na **Figura 8**. Esse processo pretende determinar as palavras que caracterizam ou que possuem maior importância em um dado documento, para isso são necessários os seguintes passos:

1. Os documentos são transformados em texto plano e divididos em palavras individuais.
2. O conjunto de palavras obtido com a aplicação do passo anterior é submetido a um processo de remoção de palavras, no qual são removidas palavras que não possuem importância no texto, chamadas na literatura como *stop words*; nesse caso, são removidos artigos, numerais, pronomes e verbos.
3. Por fim, as palavras restantes passam por um processo mencionado na literatura como *word stemming*, que tem por objetivo remover variações de um mesmo termo, como por exemplo conjugações verbais. Para esse fim, podem-se usar conceitos de similaridade entre palavras, como, por exemplo, o coeficiente de *jaccard*.

Observa-se (HAN *et al*., 2006) que a dimensionalidade do documento é proporcional à quantidade de palavras que ele possui e, após a aplicação desses 3 passos, consegue-se um conjunto de palavras mais relevantes ao documento e a consequente diminuição da dimensionalidade dele, conforme mostrado em (BRÜCHER; KNOLMAYER; MITTERMAYER, 2002).

Note que, caso seja utilizado o *HADOOP* como ferramenta de tecnologia para essa análise, algumas dessas etapas, bem como algumas próximas serão agrupadas em uma única, mas é importante conhecer qual é a função de cada uma delas, conforme essa descrição.

Seleção de Feições

Após a aplicação do processo de extração de feições, é aplicado o processo de seleção de feições, que define a importância de cada termo para um documento ou para um dado conjunto de documentos, pois nem todos os termos que permaneceram na representação dos documentos agregam conhecimento.

Conforme se observa em Souza (2010), diversas métricas encontradas na literatura podem ser aplicadas; por exemplo, métodos estatísticos, entropia ou frequência dos termos. Um método comumente utilizado é o chamado TF/IDF, ou frequência do termo (tf – *term frequency*), e a inversa da frequência do documento, ou (idf – *inverse document frequency*), o seu produto é usado para determinar o poder de discriminação de uma dada palavra para um determinado documento ou conjunto de documentos (HAN *et al.*, 2006), (CALVO; LEE; LI, 2004), (ROSE, 1994).

O *tf* define a importância de uma palavra em um documento e é diretamente proporcional à quantidade de vezes que o termo aparece em um dado documento. É dado por:

$$tf_{i,j} = \frac{n_{i,j}}{\Sigma_k n_{k,j}}$$

Onde, $n_{i,j}$ é a frequência do termo/palavra *i* no documento *j*.

Observe que nem todos os termos que possuem valores de *tf* altos são importantes para todo o conjunto de documentos, pois nem todos os documentos são importantes para a análise. Para se encontrar esse valor, é necessário calcular o *idf*, dado por:

$$idf_i = \log \frac{|D|}{\left|\{d_j : t_i \varepsilon d_j\}\right|}$$

Onde, *D* é o conjunto total de documentos, $\{d_j : t_i \varepsilon d_j\}$ é o conjunto de documentos no qual o termo t_j aparece, isto é, $n_{i,j}$ diferente de 0.

Sendo assim, o valor de *tf idf* de uma única palavra é o produto entre os dois valores encontrados. Esse valor deve ser então normalizado, isso é dado por:

$$w_{ki} = \frac{tfidf\left(t_k, d_i\right)}{\sqrt{\Sigma_{i=1}^{|T_r|}\left(tfidf\left(t_k, d_i\right)\right)^2}}$$

Onde, w_{ki} é o peso atribuído ao termo t_k no documento d_j.

Com os pesos de cada palavra definidos, pode-se fazer um ranqueamento, onde as *k* feições ou palavras mais importantes para um dado documento *j* são obtidos pela seleção das *k* palavras com valores de *tf idf* ordenados (SOUZA, 2010).

Representação de Documentos

Um documento ou um padrão pode ser representado em termos das características ou feições selecionadas, transformadas em vetores de características, onde cada termo importante deve ter um valor e posição definida no documento ou conjunto de documentos.

Se o processo de seleção produzir n como quantidade de feições e m como quantidade de documentos no conjunto total, o conjunto de documentos será representado por uma matriz de feições *m X n*. Um dado conjunto *n* de feições ou características de um dado documento ou conceito é representado por *1 X n* vetor de feições representado por *f*, conforme a representação dada: $f = (f_1, f_2, f_3, \ldots, f_n)$, onde cada f_i corresponde ao *tf* do termo ou feição que *i* representa.

O conjunto total de documentos é representado por uma matriz de *m* linhas, que representam os documentos, com *n* colunas, que representam as feições. Observe que, a partir de um conjunto de documentos representados por vetores de feições, pode-se obter uma comparação, usando uma métrica adequada para se encontrar a similaridade entre eles, como, por exemplo, a distância euclidiana.

Note que a implementação desses diversos passos já está presentes em algumas plataformas de *Big Data* ou de análise de textos, como, por exemplo, *HADOOP* com *Mahout*, dentre outras tecnologias, não havendo a necessidade de se preocupar com a codificação, embora seja importante conhecer quais métricas são utilizadas.

Nesse ponto, já se possui a valoração e caracterização matemática de todos os documentos e termos, bem como os dicionários de dados a serem utilizados. Com base nessas matrizes, é possível ir ao passo de criar ou utilizar efetivamente os algoritmos de classificação ou de agrupamento, ou apenas, utilizar os termos principais em uma análise não algorítmica, como, por exemplo, simplesmente criar uma nuvem de palavras que expressa um documento, um autor, ou um conjunto de documentos.

Exemplo Prático de Análise de Textos com Weka

Para o exemplo prático de utilização do TF/IDF, vamos utilizar o *software Weka*, vamos usar a base de dados de SPAMs, que é uma base livre para pesquisas e experimentação.

Conforme se observa na **Figura 9**, para se trabalhar com textos no *Weka*, tem-se que aplicar filtros.

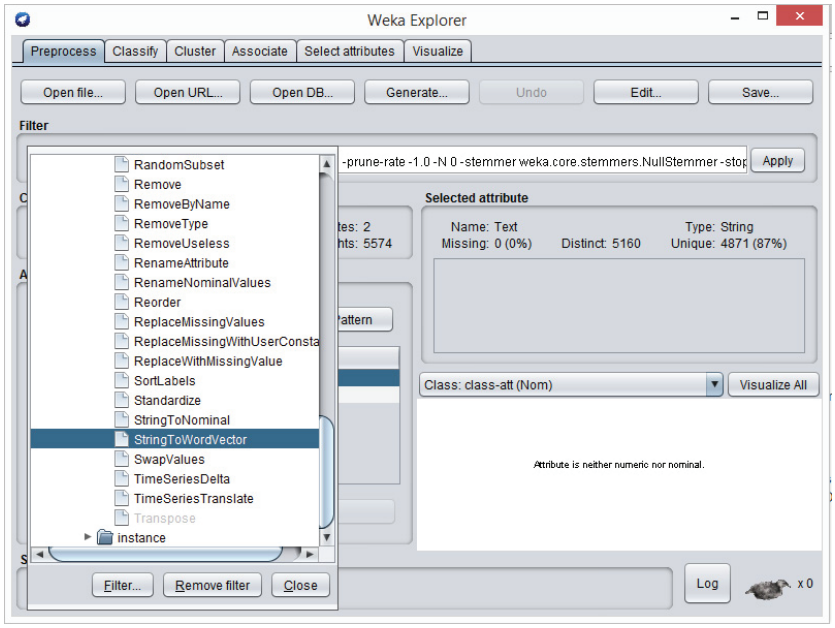


Figura 9

O filtro utilizado será o *StringToWordVector*, que tem a função de analisar os textos e criar os vetores com os valores de cada palavra. A **Figura 10** ilustra os parâmetros que podem ser passados ao filtro específico. Foi selecionado o filtro "*Weka, Filtters, unsupervised, attribute*".

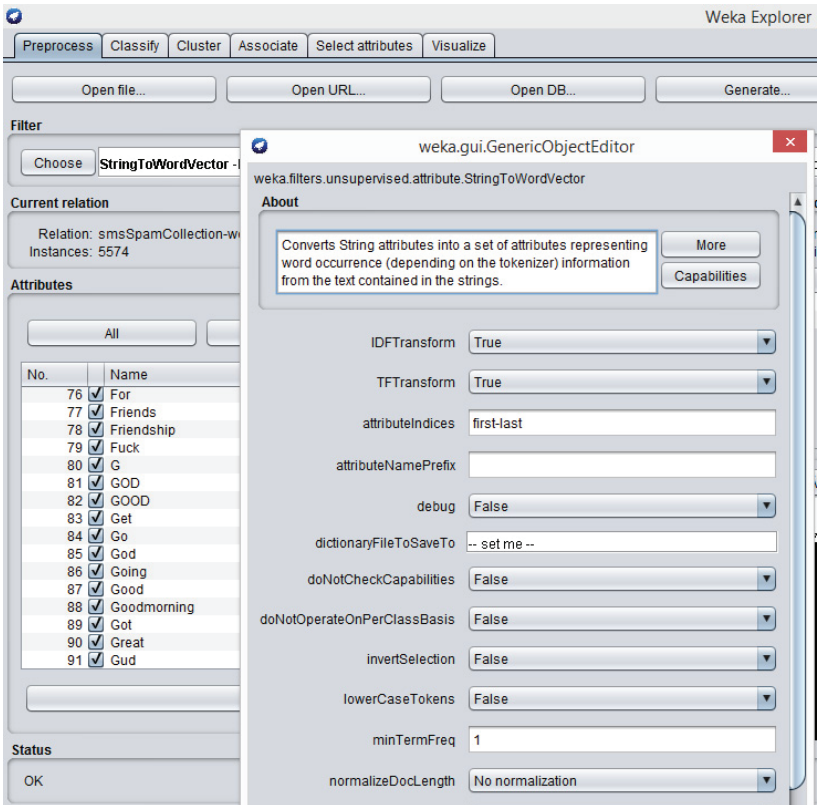


Figura 10

Note que para esse filtro são selecionadas as transformações TF/IDF; após se aplicarem os filtros, o *software* já irá calcular os valores para cada termo do conjunto total de palavras e documentos existentes na base.

Após a aplicação dos filtros, é possível então se executar os algoritmos, como, por exemplo, o algoritmo *Kmeans*, conforme ilustra a **Figura 11**.

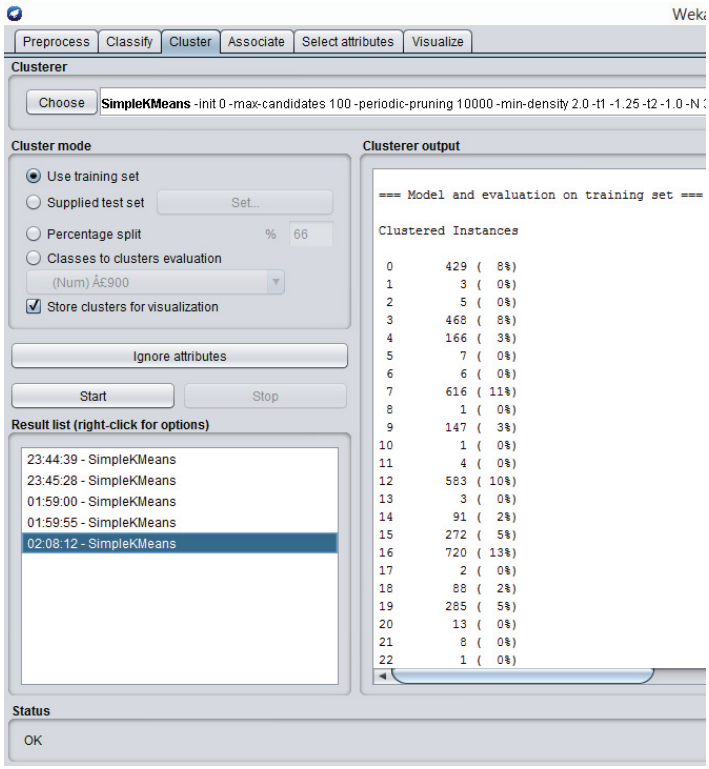


Figura 11 – Execução do algoritmo *kmeans* com base de textos no Weka.

Observe que os algoritmos aqui mencionados possuem implementações em outras plataformas, inclusive com implementações em plataformas de *Big Data*, como o *Mahout* ou *Spark*, dentre outras.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Leitura

Data Mining* na Prática: Algoritmo *K-Means

Veja neste artigo como utilizar um algoritmo clássico de classificação (*clustering*) para segmentação de dados de acordo com categorias.

https://goo.gl/RFJTji

Algoritmo *K-means:* Aprenda essa técnica essêncial através de exemplos passo a passo com *Python*

https://goo.gl/WFveKh

Mineração de Texto: Entenda a importância e quais as suas principais técnicas

https://goo.gl/VvDhp1

Tutorial: Finding Important Words in Text Using TF-IDF

Another TextBlob release (0.6.1, changelog), another quick tutorial. This one's on using the TF-IDF algorithm to find the most important words in a text document. It's simpler than you think.

https://goo.gl/pVVJ2Q

Referências

DUDA, R. O.; HART, P. E.; STORK, D. G. ***Pattern Classification***. 2. ed. *Wiley-Interscience*, 2000. ISBN: 0471056693.

THEODORIDIS, S.; KOUTROUMBAS, K. ***Pattern Recognition***, 4. ed. *Academic Press*, 2008.

SOUZA, A. M. da C. S. **Um método para predição de ligações a partir de mineração em textos e métricas em redes sociais**. Tese (mestrado em engenharia eletrônica e computação) – Instituto Tecnológico de Aeronáutica, São José dos Campos, 2010. Disponível em: <http://www.bd.bibl.ita.br/tesesdigitais/verifica_session.php?num_tese=59903&origem=BDITA>. Acesso em 2017-03-07.


Cruzeiro do Sul
Educacional